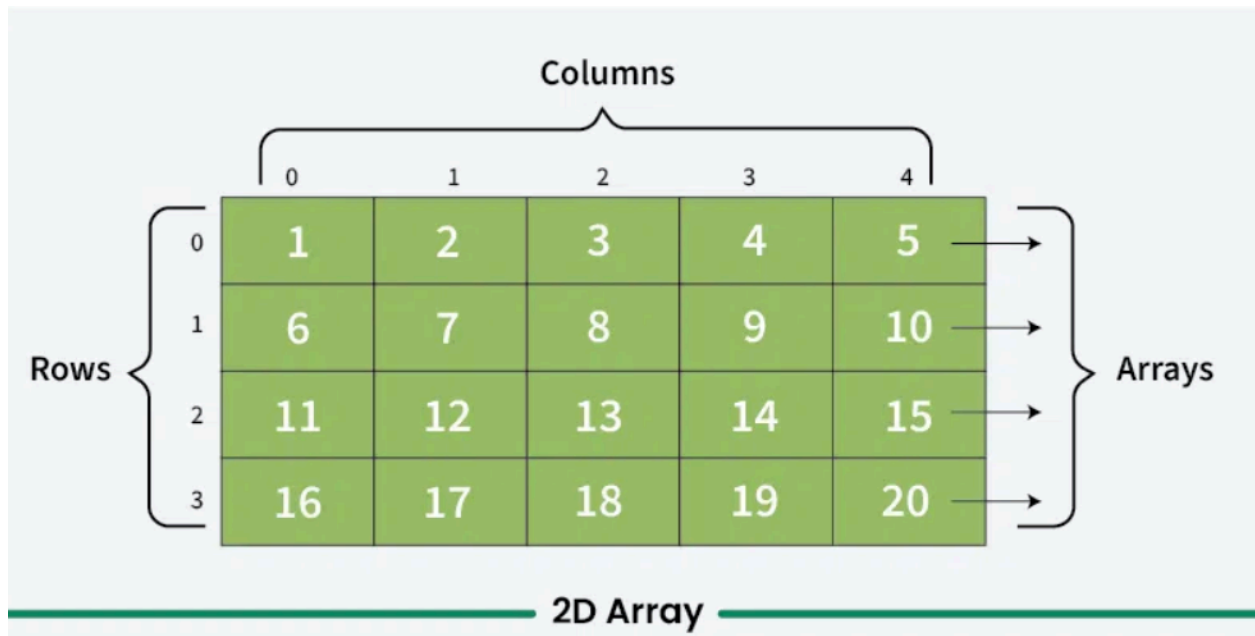




```
int arr[3][2] = {
    { 0, 1 }, { 2, 3 }, { 4, 5 }
};

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        printf("arr[%d][%d]: %d  ", i, j, arr[i][j]);
    }
    printf("\n");
}

return 0;
}
```

--Sum of Elements

```
#include <stdio.h>
```

```
int main() {
    int arr[3][2] = { {0, 1}, {2, 3}, {4, 5} };
}
```

```
int sum = 0;

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        sum += arr[i][j];
    }
}

printf("Sum of elements: %d\n", sum);
return 0;
}
```

Find the Maximum Element

```
#include <stdio.h>

int main() {
    int arr[3][2] = { {0, 1}, {2, 3}, {4, 5} };
    int max = arr[0][0];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            if (arr[i][j] > max) {
                max = arr[i][j];
            }
        }
    }

    printf("Maximum element: %d\n", max);
    return 0;
}
```

```

#include <stdio.h>

int main() {
    int arr[3][2] = { {0, 1}, {2, 3}, {4, 5} };
    int evenCount = 0, oddCount = 0;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            if (arr[i][j] % 2 == 0) {
                evenCount++;
            } else {
                oddCount++;
            }
        }
    }

    printf("Even numbers: %d\nOdd numbers: %d\n", evenCount, oddCount);
    return 0;
}

```

Find element with Position:

```

#include <stdio.h>

#define ROWS 3
#define COLS 2

void findElementPosition(int arr[ROWS][COLS], int target) {
    int found = 0;

```

```
for (int i = 0; i < ROWS; i++) {  
    for (int j = 0; j < COLS; j++) {  
        if (arr[i][j] == target) {  
            printf("Element %d found at position (%d, %d)\n", target, i, j);  
            found = 1;  
            break;  
        }  
    }  
}  
if (!found) {  
    printf("Element %d not found in the array.\n", target);  
}  
}
```

```
int main() {  
    int arr[ROWS][COLS] = { {0, 1}, {2, 3}, {4, 5} };  
    int target;  
  
    printf("Enter the element to search for: ");  
    scanf("%d", &target);  
  
    findElementPosition(arr, target);  
  
    return 0;  
}
```

3D Array:

```
int arr[2][3][2] = { { { 1, 1 }, { 2, 3 }, { 4, 5 } },  
                    { { 6, 7 }, { 8, 9 }, { 10, 11 } } }
```

// C program to print elements of a Three-Dimensional Array

```
#include <stdio.h>
```

```
int main() {
```

```
    // Create and Initialize the 3-dimensional array
```

```
    int arr[2][3][2] = { { { 1, 1 }, { 2, 3 }, { 4, 5 } },  
                        { { 6, 7 }, { 8, 9 }, { 10, 11 } } };
```

```
    // Loop through the depth
```

```
    for (int i = 0; i < 2; ++i) {
```

```
        // Loop through the rows of each depth
```

```
        for (int j = 0; j < 3; ++j) {
```

```
            // Loop through the columns of each row
```

```
            for (int k = 0; k < 2; ++k)
```

```
                printf("arr[%i][%i][%i] = %d  ", i, j, k,
```

```
                    arr[i][j][k]);
```

```
            printf("\n");
```

```
        }
```

```
    printf("\n\n");
```

```
}  
  
return 0;  
  
}
```

Character array:

```
#include <stdio.h>
```

```
int main() {  
  
    char arr[5] = {'a', 'b', 'c', 'd', 'f'};  
    int length = 5; // Length of the array  
  
    // Print the original array  
    printf("Original array: ");  
    for (int i = 0; i < length; i++) {  
        putchar(arr[i]); // Print each character  
    }  
    printf("\n");  
  
    // Print the reversed array  
    printf("Reversed array: ");  
    for (int i = length - 1; i >= 0; i--) {  
        putchar(arr[i]); // Print each character in reverse order  
    }  
    printf("\n");  
}
```

```
    return 0;
}
```

Pointer:

```
#include <stdio.h>

int main()
{
    int var = 5;
    printf("var: %d\n", var);

    // Notice the use of & before var
    printf("address of var: %p", &var);
    return 0;
}
```

```
-----
var: 5
address of var: 2686778
```

```
int* p;
int *p1;
int * p2;

int* p1, p2;
int* pc, c;
```

```
c = 5;
pc = &c;
```

```
int* pc, c;
c = 5;
pc = &c;
printf("%d", *pc); // Output: 5
```

```
int* pc, c;
c = 5;
pc = &c;
*pc = 1;
printf("%d", *pc); // Ouptut: 1
printf("%d", c); // Output: 1
```

```
#include <stdio.h>
int main()
{
    int* pc, c;

    c = 22;
    printf("Address of c: %p\n", &c);
    printf("Value of c: %d\n\n", c); // 22

    pc = &c;
```



```
printf("Address of pointer pc: %p\n", pc);  
printf("Content of pointer pc: %d\n\n", *pc); // 22
```

```
c = 11;  
printf("Address of pointer pc: %p\n", pc);  
printf("Content of pointer pc: %d\n\n", *pc); // 11
```

```
*pc = 2;  
printf("Address of c: %p\n", &c);  
printf("Value of c: %d\n\n", c); // 2  
return 0;  
}
```

Address of c: 2686784

Value of c: 22

Address of pointer pc: 2686784

Content of pointer pc: 22

Address of pointer pc: 2686784

Content of pointer pc: 11

Address of c: 2686784

Value of c: 2